



ANKARA YILDIRIM BEYAZIT UNIVERSITY
GRADUATE SCHOOL OF NATURAL AND APPLIED
SCIENCES
COMPUTER ENGINEERING

MatGen: A Realistic Sparse Matrix Generator

Final Report

By

Ali Emre Pamuk 21050111021

Faruk Kaplan 21050111026

Mert Altekin 21050111065

Supervised by

Asst. Prof. Fahreddin Şükrü Torun

Date

16.05.2025

Abstract

Sparse matrices are foundational to numerous scientific and engineering applications, yet realistic synthetic generation of such matrices remains a significant challenge. This project introduces MatGen, a comprehensive framework for generating, scaling, and optimizing realistic sparse matrices that resemble those used in simulation tools like ANSYS. We explore a broad suite of generation techniques—including interpolation, signal processing, graph-based models, and a custom autoregressive neural network architecture—to synthesize structurally and numerically similar matrices.

To further enhance realism, we introduce a local optimization strategy based on property-driven loss minimization. Performance was evaluated using feature-based loss metrics and cosine similarity, both before and after optimization. Our results show consistent improvements in matrix alignment after refinement, despite constraints in hardware and training time. Bilinear and Fourier methods were excluded from evaluation due to their inefficiency at scale.

The system is supported by a user-friendly GUI built with Flask, enabling flexible interaction with matrix generation and neural network components. Looking forward, future work includes expanding training iterations, improving model complexity, incorporating principal component analysis (PCA) for more robust similarity metrics, and transitioning to more powerful hardware for scaling experiments.

Contents

Introduction	3
Methodology	4
Data Collection	4
Matrix Generation	6
Bilinear Interpolation Method	7
Nearest Neighbor Method	7
Gaussian Pyramid Method	8
Lanczos Resampling Method	9
Wavelet Transform Method	10
Discrete Cosine Transform Method	11
Fourier Transform Method	12
Graph-Coarsening Method	13
Kronecker Product Method	14
Optimization	15
Neural Networks	16
Graphical User Interface (GUI) Development	17
Challenges and Limitations	19
Matrix Collection and Preparation	19
Feature Engineering and Property Calculation	19
Matrix Generation and Optimization	19
Computational and Resource Constraints	19
Graphical User Interface (GUI) Development	20
Application-Specific Issues	20
Results	21
Future Work	23
Improving Neural Network Capability	23
Feature Selection and Analysis	23
Code Quality and Modularity	23
Numerical Property Calculation	23
Optimization Improvements	23
Cosine Similarity Refinement	23
Model Training and Scaling	24
Hardware and Infrastructure	24
References	25
Appendix	26

Introduction

Sparse matrices play a crucial role in modern science and engineering, particularly in fields such as mechanical engineering, medicine, scientific simulations, data analysis, and large-scale numerical modeling. These matrices, characterized by a high proportion of zero-valued elements, allow for significant improvements in memory efficiency and computational speed when properly utilized. Their sparse structure enables scalable processing of extremely large systems, making them indispensable in applications ranging from medical imaging to simulating complex structure durability.

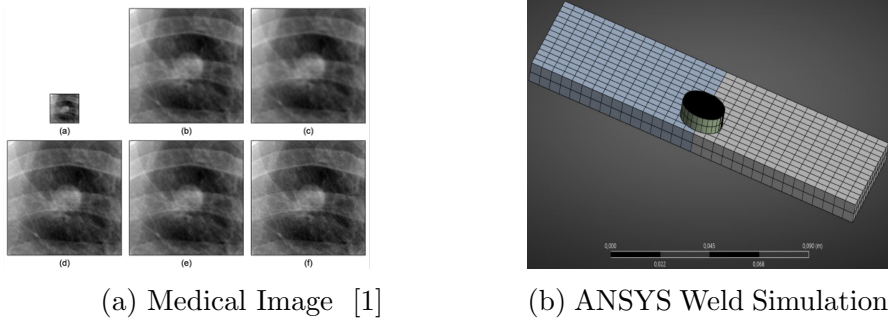


Figure 1: Example Use Cases

Despite their advantages, the generation and manipulation of sparse matrices present several challenges. Ensuring properties such as structural sparsity, numerical stability, symmetry, and realism in synthetic matrices is not straightforward. Most real-world sparse matrices contain complex patterns and dependencies that are difficult to replicate using naïve or random generation methods. Additionally, traditional generation techniques often fall short in mimicking the structure and behavior of matrices observed in real datasets, which can impact the performance and accuracy of algorithms relying on them.

This project aims to address these challenges by introducing a comprehensive framework for the realistic generation, manipulation, and optimization of sparse matrices. The proposed system focuses on generating matrices of varying sizes and complexities while preserving essential mathematical and structural features. By integrating both classical numerical strategies and cutting-edge artificial intelligence (AI) techniques, the framework offers a versatile and scalable toolset for sparse matrix synthesis.

This project builds upon key advancements in sparse matrix generation, such as the R-MAT graph generator [2], GPU-based matrix assembly techniques [3], and classical toolkits like SPARSKIT [4].

Through this work, the project contributes to the advancement of matrix-based computational systems by offering a solution that balances efficiency, realism, and adaptability. The research is grounded in the analysis of foundational works in sparse matrix theory and generation, and further extends current capabilities by incorporating AI-driven and graph-based approaches to simulate more realistic and useful matrix structures.

This work sets the stage for addressing sparse matrix challenges with practical and theoretical tools. The next section outlines the methodology driving these contributions.

Methodology

The methodology of this project is structured around two core phases: data collection and method implementation. The initial phase focuses on gathering a diverse and representative set of sparse matrices from publicly available datasets and simulation tools to guide the design and validation of the generation methods. These datasets provide valuable insight into the structural characteristics and complexities found in real-world sparse matrices across various domains.

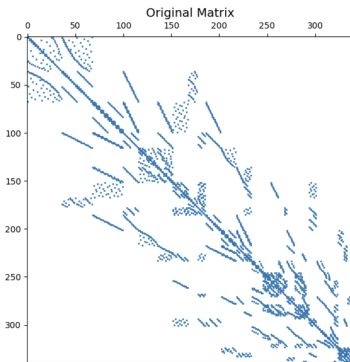
Following data acquisition, the second phase involves the design and development of a modular matrix generation framework. Each method was implemented as an independent Python module, utilizing mathematical constructs, signal transformations, and graph-based models to synthesize realistic sparse matrices. By combining empirical data analysis with modular algorithmic design, the framework ensures both flexibility and scalability, supporting a wide range of application-specific requirements.

Data Collection

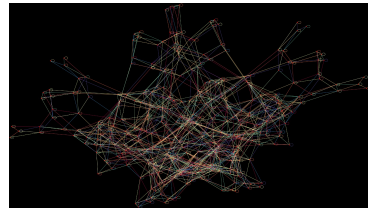
To ensure that the implemented methods could effectively mimic real-world matrix structures, a diverse set of sparse matrices was collected and analyzed prior to development. The data collection process combined publicly available datasets with custom simulations to represent a wide range of domains and complexity levels.

First, several benchmark datasets were gathered from prominent online repositories:

- **SuiteSparse Matrix Collection**, which offers matrices from scientific computing and real-world applications such as structural engineering and computational fluid dynamics.
- **Stanford Network Analysis Project (SNAP)**, providing graph-based sparse matrices from large-scale social and information networks.
- **National Institute of Standards and Technology (NIST)**, featuring standardized test matrices
- **GitHub repositories**, containing matrices published as part of open-source scientific projects and academic studies.



(a) Cage7 [5]



(b) Graph of Cage7 [5]

Figure 2: Cage7 Matrix From SuiteSparse Matrix Collection

In addition to static datasets, a custom simulation script was developed using the scikit-fem library, a Python-based finite element analysis toolkit. This script enabled the generation of parametric sparse matrices under controlled physical conditions, allowing the exploration of structural variations not present in the downloaded datasets 1 2 3 4 5 6 7 8.

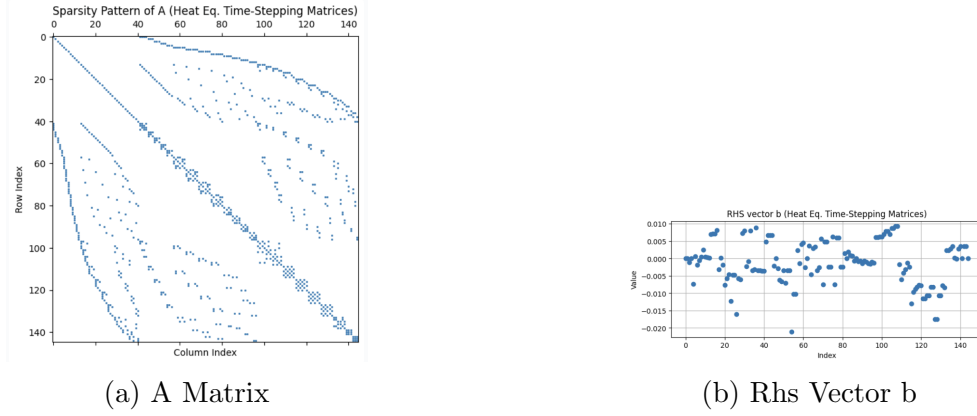


Figure 3: Heat Transfer Simulation by Scikit-Fem

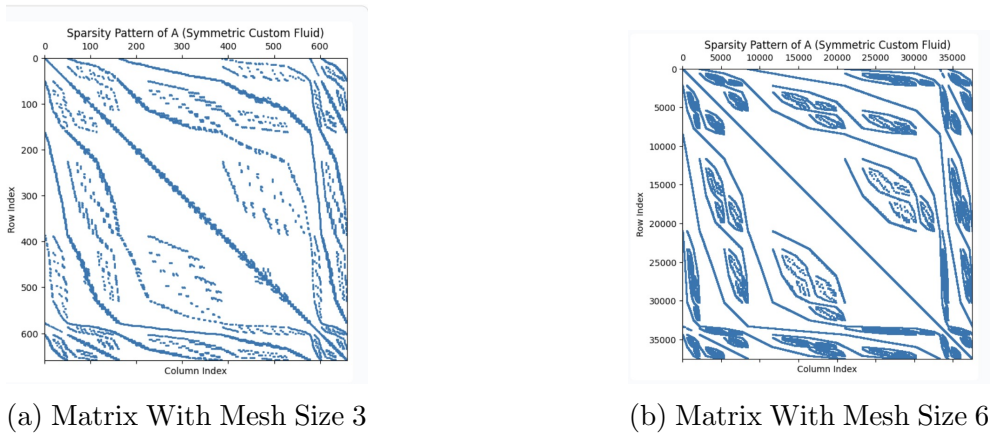


Figure 4: Symmetric Fluid Simulation With Different Mesh Sizes

Lastly, real-world finite element analysis (FEA) data was obtained from a published study [6]. The corresponding simulation was integrated into the project and sparse matrix extraction process was automated using an APDL script[7]. These simulations provided complex and application-specific sparse matrices derived from structural analysis scenarios, further enriching the dataset used for evaluation and inspiration.

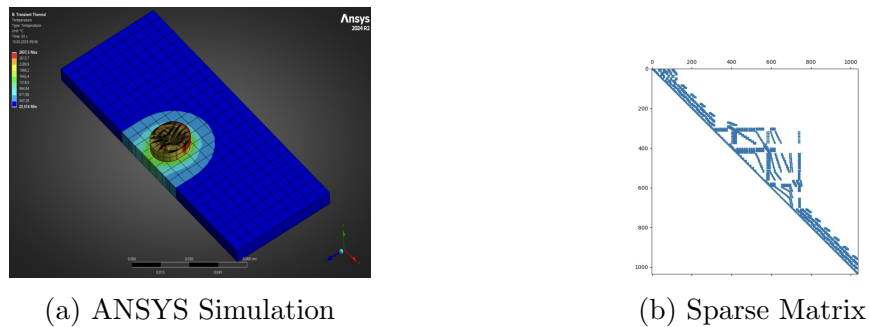
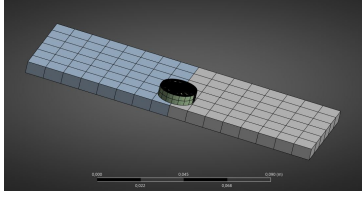
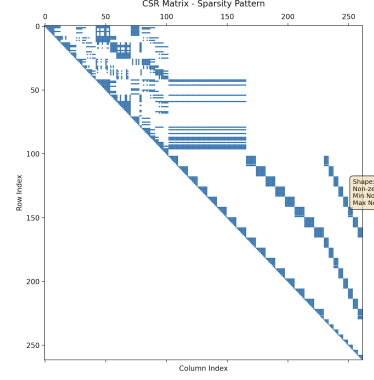


Figure 5: Weld Simulation in ANSYS

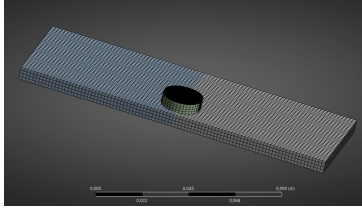


(a) Simulation With Mesh
Size = 0.1 m

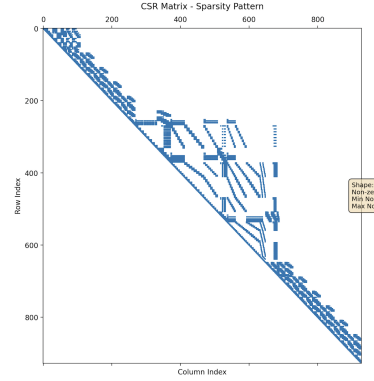


(b) Resulting Sparse Matrix

Figure 6: Weld Simulation in ANSYS



(a) Simulation With Mesh
Size = 0.0105 m



(b) Resulting Sparse Matrix

Figure 7: Weld Simulation in ANSYS

Matrix Generation

This section outlines the core matrix generation techniques developed for this project. Each method is designed to produce realistic sparse matrices with specific structural or statistical properties inspired by real-world data sources.

To provide modularity and coverage across different application domains, the implemented methods are grouped into four major categories:

- **Image-Based Methods:** Techniques that generates sparse matrices by using specialized image-based methods.
- **Signal-Based Methods:** Approaches based on signal processing that operates in frequency domain.
- **Graph-Based Methods:** Matrix generation using graph theory, adjacency matrices, and stochastic models to mimic network-like structures.
- **Neural Network-Based Methods:** Learning-inspired models capable of generating or transforming sparse matrices based on latent representations.

We will start with image processing methods

Bilinear Interpolation Method

The bilinear interpolation method is employed to scale sparse matrices to different dimensions while preserving structural continuity and sparsity characteristics. This technique is particularly useful in scenarios where the size of a sparse matrix must be increased without significantly altering its underlying distribution or density.

The method operates by sampling a set of coordinates in the new matrix space and estimating their values based on weighted contributions from the four nearest neighbors in the original matrix. Given a target point (x, y) in the scaled matrix, its corresponding location in the original matrix is determined by scaling:

$$x' = \frac{x}{s}, \quad y' = \frac{y}{s}$$

where s is the scale factor defined as:

$$s = \frac{\text{new_size}}{\text{original_size}}$$

The interpolation is then performed using the values at the four neighboring grid points: Lower-left, lower-right, upper-left and upper-right neighbors. Then, the final interpolated value $V(x, y)$ is computed as:

$$\begin{aligned} V(x, y) = & (1 - \alpha)(1 - \beta) \cdot f(x_0, y_0) + \alpha(1 - \beta) \cdot f(x_1, y_0) \\ & + (1 - \alpha)\beta \cdot f(x_0, y_1) + \alpha\beta \cdot f(x_1, y_1) \end{aligned}$$

Where:

- $\alpha = x' - x_0$
- $\beta = y' - y_0$
- $f(x_i, y_j)$ represents the matrix value at that coordinate.

To maintain the sparsity pattern, only non-zero interpolated values above a threshold (1×10^{-10}) are retained. Moreover, a random sampling strategy is applied to ensure a balanced spatial distribution of non-zero elements. An optional parameter, `match_nnz`, ensures that the output matrix contains approximately the same number of non-zero elements as the original, thus preserving density.

This method provides an effective way to generate realistic large-scale sparse matrices from smaller inputs, making it ideal for testing and benchmarking in data-scarce environments 9.

Nearest Neighbor Method

When scaling, the original non-zero coordinates are adjusted according to the scale factor:

$$s = \frac{\text{new_size}}{\text{original_size}}$$

$$x' = \lfloor x \cdot s \rfloor, \quad y' = \lfloor y \cdot s \rfloor$$

To fill in additional non-zero elements and mimic clustered patterns often seen in real-world matrices, new entries are generated around base points within a defined neighborhood radius. This is done by randomly offsetting the row and column indices of selected base points. Value assignment for these new points is sampled from the original matrix’s non-zero values, ensuring the preservation of magnitude distribution.

To avoid overwriting and ensure uniqueness, overlapping entries are resolved by keeping the maximum value at each location. The method supports both downscaling and upscaling and optionally enforces a target number of non-zero elements to match the density of the original matrix.

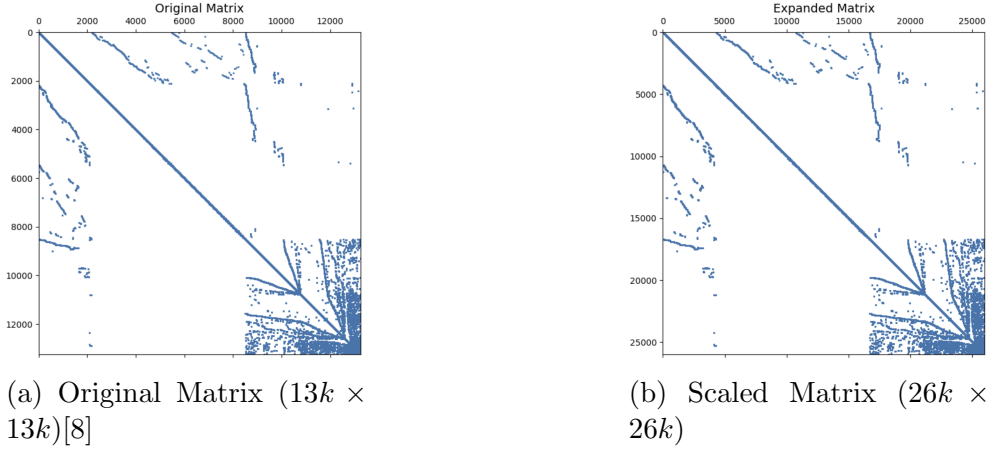


Figure 8: Matrix Generation Using Nearest Neighbor Interpolation Method

This approach is particularly well-suited for preserving discrete or clustered sparsity structures in matrices derived from image-like or spatially localized data [10].

Gaussian Pyramid Method

The Gaussian pyramid method is used for progressive downsampling of sparse matrices, primarily in cases where structural smoothing and resolution reduction are desired. Inspired by image processing techniques, this method iteratively blurs and reduces the matrix while preserving sparsity.

At each stage, a Gaussian blur is applied using a separable 2D Gaussian kernel:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This kernel is implemented as the outer product of two 1D kernels, and applied only to non-zero regions to maintain computational efficiency. The blurred values are accumulated using weighted contributions from their local neighborhoods.

After blurring, downsampling is performed by retaining only elements at even-numbered row and column indices. This process effectively halves the matrix dimensions while preserving its overall structure.

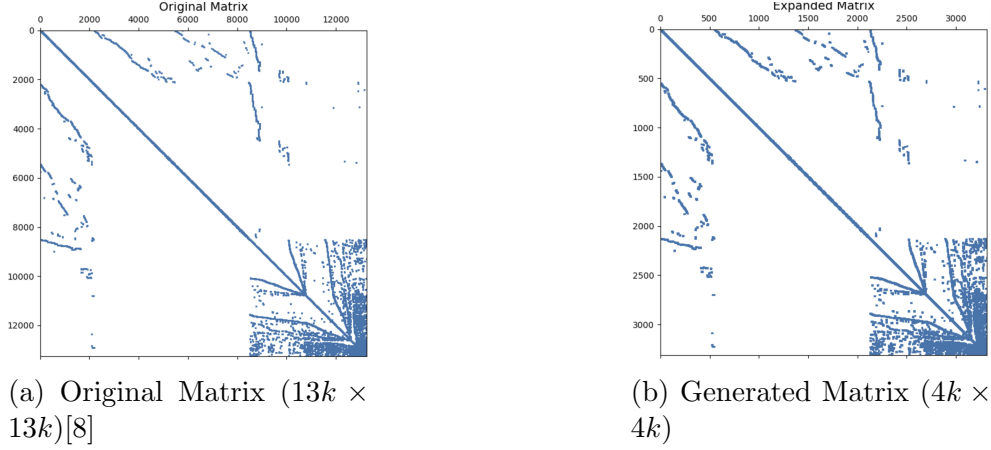


Figure 9: Matrix Generation Using Gaussian Pyramid Method

The full pyramid operation is repeated until the matrix reaches the desired target size. The sparsity is maintained by pruning negligible values (below a threshold ϵ after each iteration. This method is particularly useful for mimicking how resolution and structure degrade in physical models or multiscale systems [11].

Lanczos Resampling Method

The Lanczos method provides high-quality sparse matrix scaling by using a sinc-based interpolation kernel. Unlike simpler techniques such as nearest neighbor or bilinear interpolation, Lanczos resampling offers smoother and more accurate reconstructions, especially for preserving fine structural detail.

The core of this method is the Lanczos kernel:

$$L(x) = \begin{cases} \frac{a \cdot \sin(\pi x) \cdot \sin\left(\frac{\pi x}{a}\right)}{(\pi x)^2}, & \text{if } |x| < a \\ 1, & \text{if } x = 0 \\ 0, & \text{otherwise} \end{cases}$$

where a is the window size (commonly 2 or 3), which controls the kernel's width and smoothness. The 2D kernel is formed by separable application along both dimensions.

To scale the matrix, the method:

1. Samples candidate points in the new size via scaled mapping and random offsets.
2. Performs interpolation by applying the Lanczos filter around each candidate point, considering its corresponding region in the original matrix.
3. Normalizes and filters small values to preserve sparsity and numerical stability.
4. Optionally matches target sparsity by adjusting the number of non-zero elements to reflect the scale factor, introducing new values via local sampling if needed.

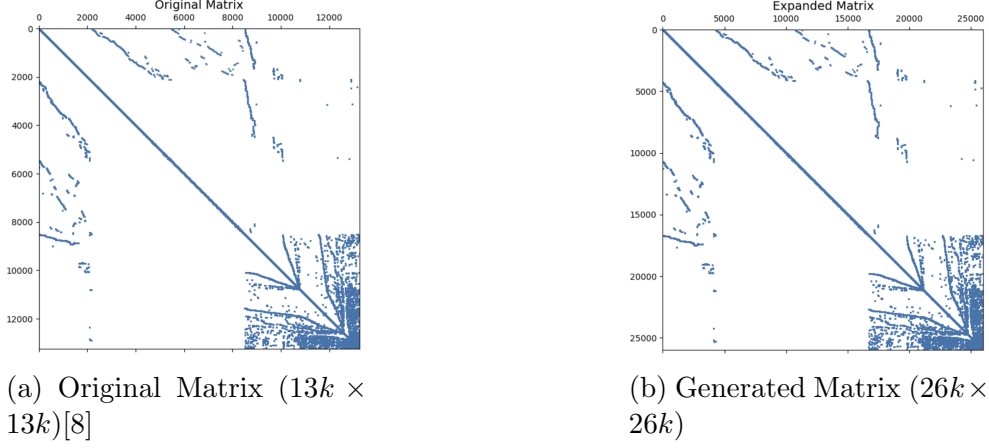


Figure 10: Matrix Generation Using Lanczos Resampling Method

This approach excels in scenarios requiring high fidelity preservation of structural patterns during upscaling, particularly where smooth transitions and minimal aliasing are critical [12].

Wavelet Transform Method

The wavelet-based method generates sparse matrices by leveraging multi-resolution analysis through 2D wavelet decomposition. This technique is especially effective for preserving localized patterns and hierarchical structures commonly observed in physical phenomena and image-derived data.

The process starts by dividing the input sparse matrix into blocks and converting each block into a dense patch. Each block is then decomposed using a discrete wavelet transform (DWT), such as Daubechies (db1) or others, up to a chosen level L . The wavelet decomposition yields one approximation matrix cA and multiple detail coefficients (cH , cV , cD) across levels.

For level-1 decomposition:

$$\text{DWT}_1(\text{block}) \rightarrow [cA, (cH, cV, cD)]$$

Each sub-band is scaled independently using interpolation, and then a reconstructed matrix is formed via the inverse wavelet transform:

$$\text{IDWT}(cA', cH', cV', cD') \rightarrow \text{rescaled block}$$

Sparse output is reconstructed by combining all rescaled blocks into a larger matrix, only preserving non-zero values above a minimal threshold.

This visual comparison highlights the effectiveness of the wavelet transform method in preserving the global structural patterns during matrix scaling. Notably, the diagonal dominance and regional sparsity of the original matrix are largely retained in the expanded output. While some minor distortions are introduced due to thresholding and interpolation artifacts, the core features remain intact, validating the applicability of wavelet-based techniques for structure-aware matrix generation.

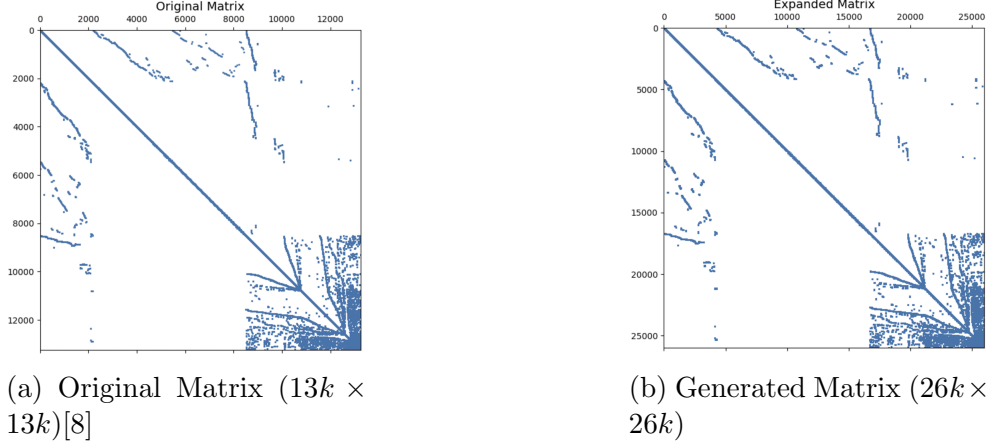


Figure 11: Matrix Generation Using Wavelet Transform Method

This method captures both global structure and localized changes in sparsity, making it highly suitable for applications requiring compression, multiscale modeling, or signal-aware sparsification [13].

Discrete Cosine Transform Method

The Discrete Cosine Transform (DCT) method is used to resize sparse matrices by operating in the frequency domain. It exploits the energy compaction property of the DCT, which concentrates important structural information into a few low-frequency coefficients. This is particularly useful for smooth transitions and compressibility in block-based systems such as signal and image processing.

The method performs the following steps:

- Blockwise decomposition: The sparse matrix is divided into dense blocks of fixed size (e.g., 8×8), and only blocks containing non-zero elements are processed.
- 2D DCT: Each block is transformed using the 2D DCT-II:

$$F(u, v) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right]$$

- Frequency-domain resizing: The DCT coefficient matrix is resized (cropped or zero-padded) according to the target scale.
- Inverse DCT: The resized coefficients are transformed back using the inverse DCT (DCT-III), reconstructing the block in the spatial domain.
- Sparsification: Values below a specified threshold are discarded to maintain sparsity.

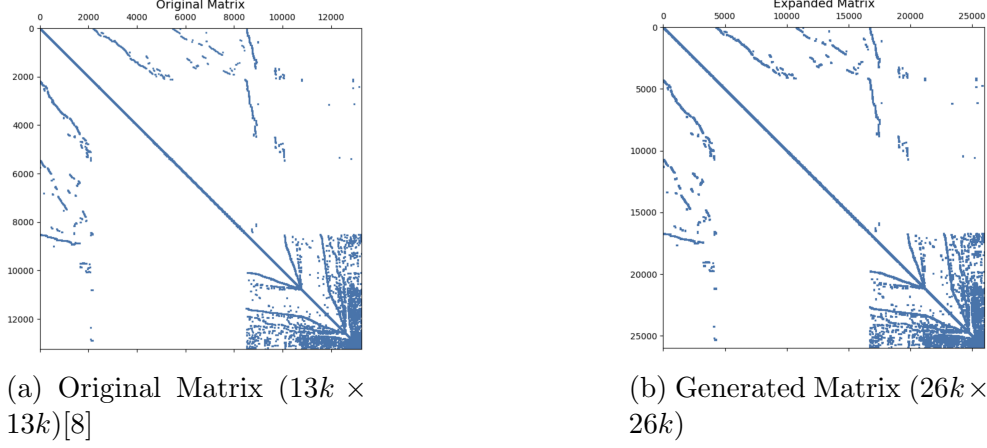


Figure 12: Matrix Generation Using Discrete Cosine Transform Method

This method is ideal for retaining global structure and smooth variation in matrix data. It’s particularly effective for compressible matrices where most energy is concentrated in low frequencies [14].

Fourier Transform Method

The Fourier-based method performs matrix scaling by manipulating the frequency domain representation of the sparse matrix. It relies on the 2D Discrete Fourier Transform (DFT), which captures global structural patterns in the data through sinusoidal basis functions.

The method proceeds as follows:

1. Forward transform: The sparse matrix is temporarily converted to a dense format and transformed using a 2D FFT:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-2\pi i \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

2. Frequency manipulation: The frequency spectrum is shifted so that low-frequency components are centered. The spectrum is then cropped or zero-padded symmetrically to match the target resolution.
3. Inverse transform: The modified frequency spectrum is inverse-transformed and shifted back to reconstruct the resized matrix in the spatial domain.
4. Sparsification: Small-magnitude values below a threshold are removed to maintain sparsity.

This method is especially well-suited for preserving global frequency characteristics and symmetric structures. However, since it requires dense FFT operations and intermediate full-matrix representations, it may introduce higher memory and computational costs, particularly for large sparse matrices. Despite this limitation, the Fourier method remains useful in scenarios where frequency integrity is crucial [15].

Graph-Coarsening Method

The graph-based method resizes sparse matrices by interpreting them as adjacency matrices and applying a multi-level coarsening and refinement scheme. This approach is particularly suited for data with inherent graph structure, such as social networks, meshes, or finite-element topologies.

The method comprises three main stages:

1. **Coarsening via Heavy-Edge Matching (HEM):** Nodes are iteratively grouped by selecting the heaviest adjacent edge (i.e., highest weight), forming coarse “super-nodes.” This procedure reduces the size of the graph while preserving dominant edge structures:

$$\text{match}(u) = \arg \max_{v \in \mathcal{N}(u)} w_{uv}, \quad \text{where } v \text{ is unmatched}$$

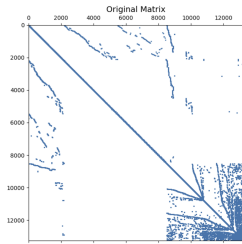
The resulting coarse matrix is formed by aggregating edge weights between matched node pairs and symmetrizing the structure.

2. **Refinement via Edge Duplication:** When upscaling, each coarse edge is probabilistically split into multiple finer connections, based on the original edge’s strength. For a coarse node pair (u, v) , s^2 fine edges may be created, where s is the split factor. Duplication probability is proportional to edge weight to preserve structure adaptively:

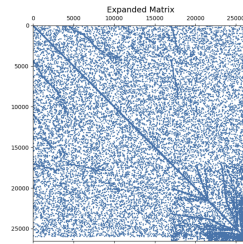
$$p_{ij} \propto \left(\frac{w_{uv}}{w_{\max}} \right)^\gamma, \quad \gamma \approx 0.2$$

3. **Optional Node Sampling and NNZ Matching:** To achieve precise matrix dimensions or match the original non-zero count, additional sampling techniques are applied:

- **Weight-aware node sampling:** favors nodes with higher strength (row sum).
- **Random injection:** injects low-weight edges into empty positions if necessary.



(a) Original Matrix $(13k \times 13k)$ [8]



(b) Generated Matrix $(26k \times 26k)$

Figure 13: Matrix Generation Using Graph Coarsening Method

This graph-based strategy is both structurally intelligent and topology-preserving, especially effective in adjacency or Laplacian-type matrices. It may incur slightly more computational overhead compared to pixel-wise methods but offers superior results for domain-specific matrices with network-like or clustered sparsity 16.

Kronecker Product Method

The Kronecker-based method generates synthetic sparse matrices by learning a small stochastic initiator matrix from the original input and expanding it through repeated Kronecker products. This technique is particularly effective for replicating large-scale, self-similar graph structures and is widely used in network modeling.

The process unfolds in three key stages:

1. 1. Initiator Learning:

The input matrix is partitioned into uniform blocks, and a probability-based initiator matrix $\Theta \in [0, 1]^{k \times k}$ is learned via block averaging:

$$\Theta_{i,j} = \frac{\text{nonzeros in block}_{i,j}}{\text{block size}_{i,j}}$$

This matrix encodes the likelihood of connections in a compressed form, capturing the structural essence of the input.

2. 2. Expansion via Kronecker Products:

To reach the target size, the learned initiator is recursively expanded:

$$\mathcal{G} = \Theta^{\otimes n}$$

The expansion is probabilistic — each entry in the resulting matrix is sampled using Bernoulli trials. To ensure consistency, the initiator is renormalized so that the expected density matches the original:

$$\mathbb{E}[\text{density}] = (\text{mean of } \Theta)^n$$

3. **Postprocessing:** If the expanded matrix overshoots the desired size, it is cropped. If sparsity does not match, additional edges are randomly added or removed. The process guarantees a final matrix of shape $(n \times n)$ with a non-zero count similar to the original.

The Kronecker product method demonstrates its ability to reproduce the global structure of the original matrix, particularly capturing diagonal sparsity patterns and large-scale symmetry. While the generated matrix may exhibit slightly reduced resolution in fine-grained features due to recursive expansion, it maintains a consistent density profile and overall layout. This makes the method especially well-suited for tasks prioritizing scalability and structural coherence.

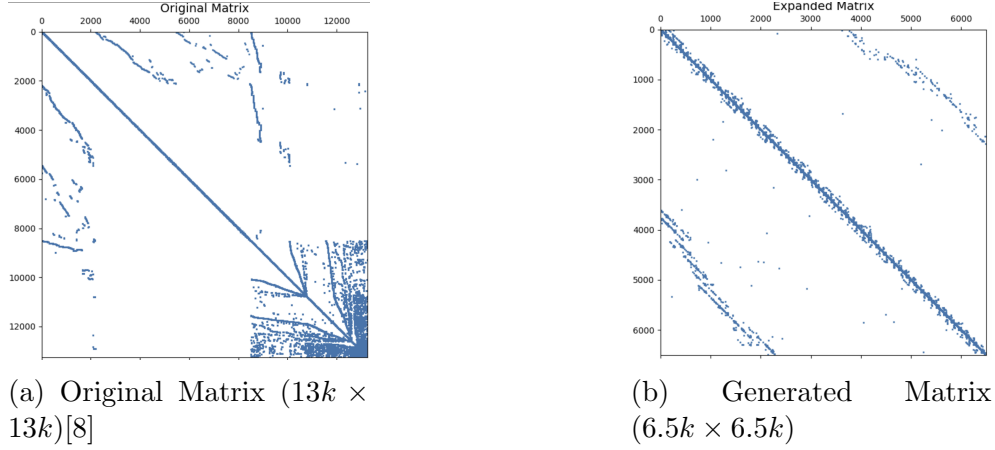


Figure 14: Matrix Generation Using Kronecker Product Method

This method offers a scalable, self-similar, and statistically controlled way of generating sparse matrices. It is also computationally efficient and memory-safe due to its strictly sparse operations. While it may introduce slight structural randomness due to its stochastic nature, it excels in modeling large, recursively structured graphs such as web networks or social graphs [17].

Optimization

To improve the realism of generated matrices, we apply a local optimization method that adjusts a single matrix X to better match the original matrix A across key numerical and structural properties.

A feature function $P(\cdot)$ computes a set of properties for any matrix:

$$P(A) = \{p_1(A), p_2(A), \dots, p_r(A)\}$$

Each property captures aspects such as:

- Row/column statistics (mean, std, median)
- Symmetry and sparsity structure
- Norms and condition estimate

We compute the loss using min-max normalized differences, weighted by property importance:

$$\text{Loss}(A, X) = \sum_{i=1}^r w_i \cdot \frac{|p_i(X) - p_i(A)|}{p_i^{\max} - p_i^{\min} + \varepsilon}$$

Weights w_i reflect each feature's relevance [2]. Normalization bounds $p_i^{\min}, p_i^{\max}$ are computed from candidate matrices.

Optimization process is as follows:

1. **Initialization:** Start with matrix X from any generation method.
2. **Iteration:**

- Apply small random changes to values or coordinates:

$$\tilde{X} = \text{Perturb}(X)$$

- Recompute loss. If improved, update

$$X \leftarrow \tilde{X}$$

3. **Output:** Optimized matrix $\tilde{X}$

This lightweight process reduces the structural and statistical gap between A and X with minimal overhead.

Neural Networks

To explore learning-based graph generation, we developed a neural pipeline centered on autoregressive modeling of sparse matrices. Our goal was to learn structural patterns directly from matrix data without intermediate lossy transformations.

We evaluated several strategies:

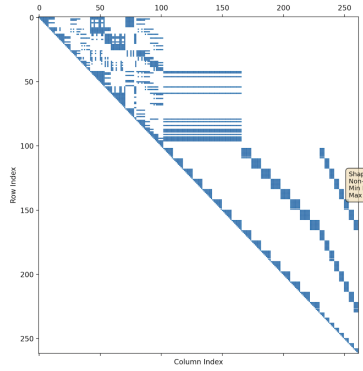
- **Image-based encoding:** Sparse matrices were treated as binary images, enabling generative fine-tuning of pretrained image models. However, this led to significant information loss, particularly in high-resolution and low-density regions.
- **Graph conversion using CoSP[9]:** We mapped matrices to graph representations and fine-tuned pretrained Graph Neural Networks (GNNs). Since actual model trained on vast amount of text data, model generated text data even after fine tuning with graph representation of sparse matrix data.
- **Custom autoregressive graph model:** Ultimately, we trained a graph autoregressive generator from scratch. Given a partial graph, the model predicts connections for the next node using a learned structural prior.

Our architecture consists of:

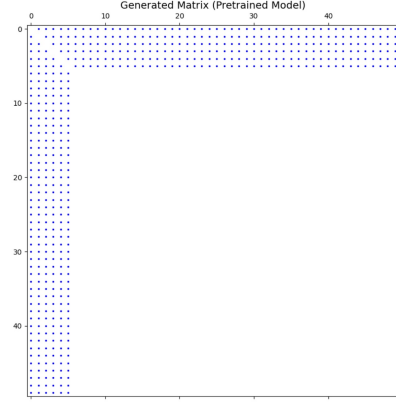
- A GraphStateUpdaterGNN that uses two GCN layers and global mean pooling to encode the current graph state.
- An AutoregressiveGraphGenerator that sequentially adds nodes and predicts edges to prior nodes via a linear edge prediction head.

All graphs were derived from ANSYS simulations and synthetically expanded to 300 training samples using our scaling methods. Each graph was represented only via structure (unweighted edges) 3.

We used true autoregressive training, where each training step involved predicting edges from a new node to its predecessors based on a partial subgraph.



(a) Overall Looking of Training Data



(b) Generated Matrix (Autoregressive Model)

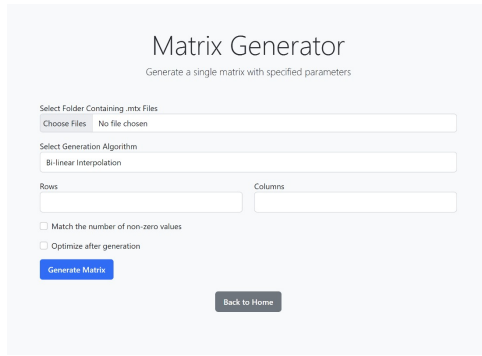
Figure 15: Matrix Generation Using Neural Networks

Graphical User Interface (GUI) Development

To streamline and simplify the workflow described in the previous sections, we developed a user-friendly Web UI. This UI serves as a comprehensive tool for single matrix generation, dataset generation, example simulation executions, and neural network operations:

- **Matrix Generation:** This module lets users upload a matrix, select the scaling method they want to use and enter the new dimensions for the new matrix and generate. After generation users can visualize matrix structures, heatmaps, graphs and numerical properties.
- **Dataset Generation:** This module lets users create datasets containing numbers of matrices they select. For generating the datasets users should specify the size of the dataset, minimum and maximum dimensions for the matrices.
- **Example Simulations:** This module lets users run example simulations prepared with scikit-fem.
- **Neural Network Operations:** This module lets users generate matrices with pre-trained models or train a new model with their choice of configuration.

The graphical user interface (GUI) was designed to streamline the matrix generation process for both individual and bulk use cases. Users can easily configure input parameters through form-based controls, enabling flexible experimentation without requiring direct code interaction. The modular layout of the GUI ensures intuitive navigation across different components of the pipeline.



Matrix Generator
Generate a single matrix with specified parameters

Select Folder Containing .mtx Files
Choose Files No file chosen

Select Generation Algorithm
Bi-linear Interpolation

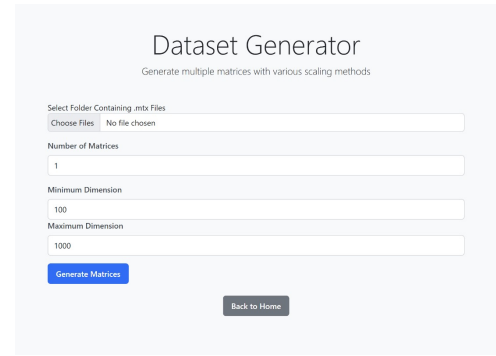
Rows Columns

☐ Match the number of non-zero values
☐ Optimize after generation

[Generate Matrix](#)

[Back to Home](#)

(a) Matrix Generation



Dataset Generator
Generate multiple matrices with various scaling methods

Select Folder Containing .mtx Files
Choose Files No file chosen

Number of Matrices
1

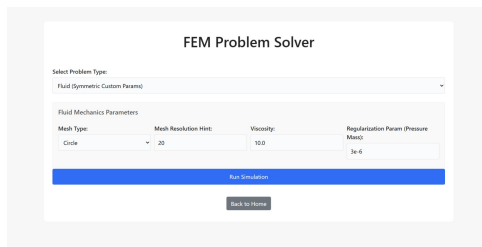
Minimum Dimension
100

Maximum Dimension
1000

[Generate Matrices](#)

[Back to Home](#)

(b) Dataset Generation



FEM Problem Solver

Select Problem Type:
Fluid (Symmetric Custom Param)

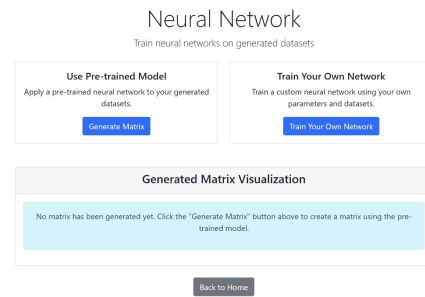
Fluid Mechanics Parameters

Mesh Type	Mesh Resolution Hint	Viscosity	Regularization Param (Pressure Mod)
Circle	20	10.0	3e-6

[Run Simulation](#)

[Back to Home](#)

(c) FEM Simulation



Neural Network
Train neural networks on generated datasets

Use Pre-trained Model
Apply a pre-trained neural network to your generated datasets.

[Generate Matrix](#)

Train Your Own Network
Train a custom neural network using your own parameters and datasets.

[Train Your Own Network](#)

Generated Matrix Visualization

No matrix has been generated yet. Click the "Generate Matrix" button above to create a matrix using the pre-trained model.

[Back to Home](#)

(d) Neural Network Operations

Figure 16: Glance to GUI

The GUI was built using Flask/HTML/CSS/JS to ensure user-friendly experience. This tool enhances accessibility, enabling both researchers and practitioners to efficiently apply the developed methodology without delving into the underlying codebase.

Challenges and Limitations

Throughout this project, we encountered various challenges, ranging from computational constraints to integration difficulties. These limitations were systematically addressed, but some persisted due to resource and platform limitations. Below, we categorize the key challenges:

Matrix Collection and Preparation

- **Collecting matrices from real-world sources:** Extracting matrices with desired properties from simulation tools such as ANSYS was time-consuming and often required manual effort.
- **Preprocessing and formatting:** Many datasets were stored in incompatible or raw formats, requiring conversion into matrix structures usable by our pipeline.

Feature Engineering and Property Extraction

- **Feature discovery and definition:** Selecting the right set of numerical and structural properties (e.g., row/column stats, symmetry, sparsity) involved significant experimentation and tuning.
- **Handling categorical and derived metrics:** Transforming structural properties into normalized numerical values suitable for optimization introduced added complexity.

Matrix Generation and Optimization

- **Balancing structure and randomness:** Generating realistic matrices that preserved structural patterns while introducing variability was non-trivial and demanded extensive parameter tuning.
- **Loss function design:** Defining and tuning a property-based loss function that could consistently guide optimization toward more realistic outputs required iterative experimentation.
- **Resource-bound optimization:** Although the optimization method was effective, it was constrained by memory and processing limits—especially in iterative fine-tuning scenarios on Google Colab.
- **Convergence difficulties:** Achieving significant loss reduction within limited time and compute budgets was a persistent challenge during refinement.

Computational and Resource Constraints

- **Hardware limitations on Colab:** Training neural models and running large matrix operations in Google Colab introduced bottlenecks due to memory caps, timeouts, and the absence of persistent storage..
- **Model performance vs. time budget:** The neural network training phase was cut short due to time restrictions on hosted environments, preventing full convergence or large-scale hyperparameter tuning.

Graphical User Interface (GUI) Development

- **Flask backend limitations:** While Flask provided a lightweight server framework, integrating it with asynchronous matrix processing and file uploads introduced complications.
- **UI responsiveness and error handling:** Ensuring smooth user experience with real-time feedback and robust error handling required custom solutions, particularly for dynamic input/output.
- **Embedding visualizations:** Adding real-time visual feedback (e.g., matrix sparsity plots, metric comparisons) in a responsive layout was technically challenging with limited frontend tooling.

Application-Specific Issues

- **Simulation tool learning curve:** Using tools like ANSYS required domain knowledge and setup effort, adding overhead to data collection.
- **Limited prior work and resources:** Few existing end-to-end frameworks tackled sparse matrix generation and optimization in this manner, necessitating custom implementation and experimentation.

Results

To assess the performance of our matrix generation methods, we evaluated each approach using two complementary metrics:

- **Property-based loss:** Measures deviation from original matrix features across key numerical and structural properties (as defined in Section X), and
- **Cosine similarity:** Evaluates global structural alignment between flattened representations of the original and generated matrices.

We report results before and after applying our optimization routine, which adjusts generated matrices locally to better match the originals.

Method	Loss (Before)	Loss (After)	Cosine Sim. (Before)	Cosine Sim. (After)	Time (s)
Nearest Neighbor	1.31e+16	1.23e+16	0.14	0.15	0.79
DCT (Blockwise)	2.02e+16	1.87e+16	0.07	0.07	13.03
Gaussian Projection	4.85e+16	4.81e+16	0.07	0.07	2.71
Wavelet Transform	5.06e+16	5.01e+16	0.11	0.12	14.83
Kronecker Model	5.07e+16	5.07e+16	0.21	0.18	6.71
Graph Coarsening	3.73e+16	3.66e+16	0.03	0.04	0.31
Lanczos Rescaling	3.02e+17	2.73e+17	0.11	0.12	54.86

Table 1: Comparison of Different Methods (Before and After Optimization)

Key findings are listed below:

- Loss consistently decreased after optimization in nearly all methods, demonstrating the effectiveness of our refinement procedure.
- Cosine similarity was preserved or modestly improved, validating structural alignment improvements.
- Kronecker model yielded the highest pre-optimization similarity (0.21), showing strong structural priors, though optimization slightly decreased its similarity.
- Nearest neighbor achieved the lowest loss and also ran very quickly, proving to be both effective and efficient.
- Graph coarsening achieved the fastest runtime (0.31s), making it the most suitable for time-constrained applications despite moderate numerical performance.
- Lanczos and wavelet methods, while numerically stable and moderately improving after optimization, had the longest runtimes (especially Lanczos at 54.86s), potentially limiting scalability.
- Gaussian projection and DCT offered balanced performance but were slower than coarsening or nearest methods.

We emphasize that all experiments were conducted under resource constraints. Optimization on large sparse matrices was particularly intensive and had to be tuned for runtime and memory efficiency. These limitations impacted iteration depth and scalability but were mitigated by careful engineering of lightweight optimization logic.

Note: Bilinear interpolation and Fourier-based resizing were also implemented, but excluded from quantitative comparison due to their high memory usage and computational inefficiency on large sparse matrices. While theoretically valid, their practical cost outweighed their utility in our context.

Despite hardware limits, our pipeline demonstrated clear improvements with post-generation optimization. The before-after comparison validates our method’s ability to refine generated matrices toward their original structure — an encouraging result that invites further research with enhanced compute resources and larger datasets.

Future Work

While this project successfully achieved its primary objectives, there are several promising directions for further development. These aim to enhance the technical depth, scalability, and overall performance of the matrix generation and optimization framework.

Improving Neural Network Capability

- We plan to expand neural approaches by training more advanced models with deeper architectures. These models will aim to capture higher-order structural relationships and generate more realistic sparse matrices from learned priors.

Feature Selection and Analysis

- Feature selection techniques will be employed to prioritize the most significant matrix properties, reducing computational overhead while maintaining alignment quality. Properties with lower predictive impact will be down-weighted or excluded from the loss calculation.

Code Quality and Modularity

- The implementation will be further modularized for improved maintainability, execution speed, and memory efficiency. This includes refactoring components for parallel processing and simplifying integration with external simulation tools.

Numerical Property Calculation

- Numerical feature extraction will be refined to ensure robustness, especially for edge cases and ill-conditioned matrices. Enhancing precision and consistency in these calculations will support more accurate comparisons during optimization.

Optimization Improvements

- The cost function will be updated to incorporate more nuanced comparisons between generated and reference matrices, including new structural and spectral metrics.
- More deterministic and reproducible optimization strategies will be explored, reducing dependence on randomness while maintaining flexibility in local adjustments.

Cosine Similarity Refinement

- To obtain more stable and interpretable similarity measures, future work will include principal component analysis (PCA) on sparse matrices. This dimensionality reduction will help highlight dominant structural features and allow cosine similarity to be computed in a lower-dimensional, noise-suppressed subspace.

Model Training and Scaling

- Future iterations will include longer training cycles and more iterations to improve convergence, especially for neural models.
- Experiments will be extended to larger datasets with better generalization capabilities, enabled by optimized data pipelines.

Hardware and Infrastructure

- Limitations of current hardware (e.g., Google Colab constraints) restricted the scale and complexity of models. Future work will leverage more powerful computational resources to enable deeper networks, larger batch sizes, and faster optimization passes.

References

- [1] Volker Bernhard Scholz and Andreas Dorner. Big data and business intelligence: Debunking the myths. *Open Journal of Business and Management*, 5(1):20–30, 2017.
- [2] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. R-mat: A recursive model for graph mining. *SIAM International Conference on Data Mining*, 2004. Accessed from foundational works on graph mining.
- [3] Andrzej Dziekonski, Piotr Sypek, Andrzej Lamecki, and Mirosław Mrozowski. Finite element matrix generation on a gpu. *Computer Methods in Applied Mechanics and Engineering*, 254:224–232, 2013. Exploration of GPU-based matrix generation.
- [4] Youcef Saad. *SPARSKIT: A Basic Tool Kit for Sparse Matrix Computations*. University of Minnesota, 1994. Toolkit for handling sparse matrix computations.
- [5] Timothy A. Davis and Yifan Hu. The university of florida sparse matrix collection. <https://sparse.tamu.edu/vanHeukelum/cage7>, 2011. Matrix: cage7, contributed by van Heukelum.
- [6] Yasin Sarikavak. An advanced modelling to improve the prediction of thermal distribution in friction stir welding (fsw) for difficult to weld materials. *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, 43(4), 2021.
- [7] Ali Emre Pamuk. Apdl automation script for sparse matrix generation (matgen). <https://github.com/aliemrepmk/MatGen---A-Realistic-Sparse-Matrix-Generator/blob/Final/final/apdl.py>, 2025. Accessed May 2025.
- [8] N. Martins. Rommes/ww_vref_6405: Brazilian interconnect power system (bips/97). https://sparse.tamu.edu/Rommes/ww_vref_6405, 2010. SuiteSparse Matrix Collection.
- [9] Ziniu Hu, Yuxiao Dong, Kuansan Wang, Kai-Wei Chang, and Yizhou Sun. Gpt-gnn: Generative pre-training of graph neural networks. *arXiv preprint arXiv:2006.15437*, 2020.

Appendix

Algorithm 1 Solve Block Preconditioned Stokes 3D

Require: Mesh type, Mesh resolution (optional)

Ensure: Velocity field, Pressure field

- 1: Generate a 3D mesh based on the input.
 - 2: Define the problem domain, finite element spaces, and basis functions.
 - 3: Assemble the Stokes equations into a matrix system.
 - 4: Apply boundary conditions.
 - 5: Solve the resulting linear system using a preconditioned Krylov method.
 - 6: Extract velocity and pressure from the solution.
 - 7: (Optional) Compute and output the pressure error.
 - 8: (Optional) Save the results to a VTK file.
 - 9: **return** Mesh, Basis functions, Velocity, Pressure
-

Algorithm 2 Solve Convection-Diffusion Problem

Require: Mesh type, Mesh resolution (optional), Diffusion coefficient (optional), Advection vector (optional), Source term (optional)

Ensure: Mesh, Basis functions, Solution

- 1: Generate a 2D mesh (triangle) based on the input mesh type ('rectangle', 'circle', or 'random') and resolution.
 - 2: Define the finite element basis on the mesh.
 - 3: Define the bilinear forms for diffusion and advection, and the linear form for the source term.
 - 4: Assemble the matrix system from the bilinear forms and the vector from the linear form.
 - 5: Apply Dirichlet boundary conditions.
 - 6: Solve the resulting linear system.
 - 7: Plot the solution.
 - 8: **return** Mesh, Basis functions, Solution
-

Algorithm 3 Solve Fluid Symmetric

Ensure: Mesh, Basis functions, Velocity, Pressure, Stream function

- 1: Generate a circle mesh.
 - 2: Define finite element spaces for velocity and pressure.
 - 3: Define basis functions for velocity and pressure.
 - 4: Assemble the Stokes matrix system (including viscosity).
 - 5: Define the body force.
 - 6: Assemble the force vector.
 - 7: Apply boundary conditions.
 - 8: Solve the linear system to obtain velocity and pressure.
 - 9: Compute the stream function.
 - 10: Plot the velocity field, pressure field, and stream function contours.
 - 11: **return** Mesh, Basis functions, Velocity, Pressure, Stream function
-

Algorithm 4 Solve Fluid Symmetric Custom

Require: Mesh type, Mesh resolution, Viscosity, Regularization parameter

Ensure: K, f, Mesh, Basis functions, Velocity, Pressure, Stream function

- 1: Generate a mesh (circle or rectangle) based on the mesh type.
 - 2: Define finite element spaces for velocity and pressure.
 - 3: Define basis functions for velocity and pressure.
 - 4: Assemble the Stokes matrix system with given viscosity and regularization parameter.
 - 5: Define the body force.
 - 6: Assemble the force vector.
 - 7: Apply boundary conditions.
 - 8: Solve the linear system.
 - 9: Compute velocity, pressure and the stream function.
 - 10: **return** K, f, Mesh, Basis functions, Velocity, Pressure, Stream function
-

Algorithm 5 Solve Fluid Non-Symmetric

Ensure: Mesh, Basis functions, Velocity, Pressure, Stream function

- 1: Generate a circle mesh.
 - 2: Define finite element spaces for velocity and pressure.
 - 3: Define basis functions for velocity and pressure.
 - 4: Assemble the Stokes matrix system with a convection term, and apply a random permutation to the matrix.
 - 5: Define the body force.
 - 6: Assemble the force vector.
 - 7: Apply boundary conditions.
 - 8: Solve the linear system to obtain velocity and pressure.
 - 9: Compute the stream function.
 - 10: Plot the velocity field, pressure field, and stream function contours.
 - 11: **return** Mesh, Basis functions, Velocity, Pressure, Stream function
-

Algorithm 6 Solve Heat Transfer

Require: Mesh type, Mesh resolution (optional), Thermal diffusivity (optional)

Ensure: Mesh, Basis functions, Temperature field

- 1: Generate mesh (circle or rectangle, with random options).
 - 2: Define basis functions.
 - 3: Assemble Laplacian (K) and mass (M) matrices; apply boundary conditions.
 - 4: Set up time-stepping; define initial condition and thermal diffusivity.
 - 5: Construct time-stepping matrices A_time and B_time; factorize A_time.
 - 6: **for** each time step **do**
 - 7: Calculate RHS; solve for new temperature field; apply boundary conditions.
 - 8: **end for**
 - 9: Print final max temperature; plot temperature field.
 - 10: **return** Mesh, Basis functions, Temperature field
-

Algorithm 7 Solve Minimal Surface Problem

Require: Mesh type, Mesh resolution (optional)

Ensure: Mesh, Basis functions, Solution

- 1: Generate mesh (circle or rectangle).
 - 2: Define basis functions; initialize solution x ; apply boundary conditions.
 - 3: Define Jacobian and RHS forms.
 - 4: **for** iteration in range(100) **do**
 - 5: Interpolate x ; assemble J and F .
 - 6: Solve $J\Delta x = -F$; update x .
 - 7: Calculate and print convergence norm.
 - 8: **if** converged **then** break.
 - 9: **end for**
 - 10: Plot J , F , and 3D solution.
 - 11: **return** Mesh, Basis, Solution
-

Algorithm 8 Solve Linear Elasticity

Require: Mesh type, Mesh resolution (optional), Young’s modulus (optional), Poisson’s ratio (optional)

Ensure: Mesh, Basis functions, Displacement field

- 1: Generate mesh (circle or rectangle).
 - 2: Define basis functions; set material properties.
 - 3: Define elasticity and traction forms.
 - 4: Assemble stiffness matrix K and force vector F .
 - 5: Apply left-edge Dirichlet BCs by condensing K and F .
 - 6: Solve for free displacement; expand to full displacement vector.
 - 7: Calculate and print max displacement.
 - 8: Plot displacement and deformed mesh.
 - 9: **return** Mesh, Basis, Displacement
-

Feature Name	Weight
row_median_min, max, mean, std	1.5
row_mean_min, max, mean, std	0.8
row_std_min, max, mean, std	0.7
col_median_min, max, mean, std	1.0
col_mean_min, max, mean, std	0.6
col_std_min, max, mean, std	0.5
pattern_symmetry, numerical_symmetry	1.2
density, avg_distance_to_diagonal	1.2, 0.7
bandwidth	0.01
norm_1, norm_inf, frobenius_norm	0.01
estimated_condition_number	0.01
num_diagonals_with_nonzeros	0.2
num_structurally_unsymmetric_elements	0.5

Table 2: Feature names and their corresponding weights used in the loss computation.

Algorithm 9 Scale Sparse Matrix Bilinearly

Require: Original sparse matrix, New size, Match non-zero count (optional)

Ensure: Scaled sparse matrix

- 1: Convert original matrix to CSR format.
 - 2: Get original size and non-zero count.
 - 3: Calculate scale factor and target non-zero count.
 - 4: Initialize a dictionary to store new coordinates and values.
 - 5: Generate a set of random coordinates in the new matrix.
 - 6: **for** each random coordinate **do**
 - 7: Interpolate value using bilinear interpolation.
 - 8: **if** interpolated value is non-zero **then**
 - 9: Store coordinate and interpolated value.
 - 10: **end for**
 - 11: **if** match non-zero count is true **then**
 - 12: Adjust the number of stored coordinates to match the target count by randomly removing or adding coordinates.
 - 13: Convert the coordinate-value dictionary to COO format.
 - 14: Create the scaled sparse matrix from the COO data.
 - 15: **return** the scaled sparse matrix.
-

Algorithm 10 Scale Sparse Matrix Nearest Neighbor

Require: Original sparse matrix, New size, Match non-zero count (optional)

Ensure: Scaled sparse matrix

- 1: Convert to COO format; get original size, coordinates, and values.
 - 2: Calculate scale factor and target non-zero count.
 - 3: Determine neighborhood size.
 - 4: **if** downscaling **then**
 - 5: Randomly select non-zeros to keep. **else if** upscaling **then**
 - 6: Keep all original non-zeros.
 - 7: Scale coordinates of selected/all points.
 - 8: **if** upscaling **then**
 - 9: Generate additional non-zeros near existing ones.
 - 10: Combine original and new points.
 - 11: Resolve coordinate collisions by keeping maximum values.
 - 12: **if** match non-zero count **then**
 - 13: Adjust number of non-zeros by randomly removing or adding points.
 - 14: Create scaled sparse matrix.
 - 15: **return** Scaled matrix.
-

Algorithm 11 Scale Sparse Matrix with Gaussian Blur

Require: Original sparse matrix, New size, Gaussian sigma (optional)

Ensure: Scaled sparse matrix

```
1: if input is not a sparse matrix then
2:   Raise error. if
3:   Initialize current matrix with original matrix.
4: while current matrix size is more than 2 times the new size do
5:   Blur current matrix using Gaussian kernel.
6:   Downsample blurred matrix.
7:   Update current matrix with downsampled matrix While
8:   Blur the current matrix.
9:   Downsample the blurred matrix to get the final matrix.
10: if final matrix size is greater than new size then
11:   Downsample the final matrix. if
12: return final matrix.
```

Component	Value
Node embedding dim	64
GCN hidden dim	128
Graph embedding output dim	256
Max nodes to generate	Set per dataset
Loss function	Binary Cross-Entropy
Optimizer	Adam (LR = 0.001)
Epochs	50
Dataset size	300 matrices
Framework	PyTorch Geometric

Table 3: Neural network architecture and training details used for matrix generation.

Algorithm 12 Scale Sparse Matrix with Lanczos

Require: Original sparse matrix, New size, Match non-zero count (optional), Lanczos parameter α (optional)

Ensure: Scaled sparse matrix

- 1: Convert original matrix to CSR format.
 - 2: Get original size.
 - 3: Calculate scale factor.
 - 4: Calculate target number of non-zeros.
 - 5: Store original value range.
 - 6: Create a dictionary for fast lookup of original non-zero elements.
 - 7: Initialize a dictionary to store new coordinates and values.
 - 8: Calculate initial sample count.
 - 9: Initialize a set of candidate coordinates.
 - 10: Sample coordinates based on original non-zeros (50
 - 11: Apply small random jitter to spread out the samples
 - 12: Add these new coordinates to the candidate set
 - 13: Sample remaining coordinates randomly across the new matrix.
 - 14: Add these coordinates to the candidate set
 - 15: **for** each candidate coordinate **do**
 - 16: Map back to original matrix coordinates.
 - 17: Apply 2D Lanczos filter to calculate the interpolated value.
 - 18: **if** interpolated value is non-zero **then**
 - 19: Store the coordinate and interpolated value. **if**
 - 20: **end for**
 - 21: Preserve value range (optional):
 - 22: Clamp negative values if original had no negatives
 - 23: Normalize to match original value range
 - 24: **if** match non-zero count **then**
 - 25: Adjust number of non-zeros by randomly removing or adding points. **if**
 - 26: Convert the coordinate-value dictionary to COO format.
 - 27: Create the final scaled sparse matrix.
 - 28: **return** the scaled sparse matrix. =0
-

Algorithm 13 Scale Sparse Matrix with Wavelet

Require: Original sparse matrix, New size, Wavelet type (optional), Decomposition level (optional)

Ensure: Scaled sparse matrix

- 1: Ensure new_size is even.
 - 2: Calculate block size based on decomposition level.
 - 3: Convert the sparse matrix to COO format to get row and column indices, and values.
 - 4: Initialize an empty dictionary to store blocks of non-zero values.
 - 5: **for** each non-zero element in the sparse matrix **do**
 - 6: Determine the block containing the element.
 - 7: Store the element's relative position and value within the block.
 - 8: **end for**
 - 9: Calculate the scaling factor.
 - 10: Initialize lists to store the row and column indices, and values of the scaled matrix.
 - 11: **for** each block of non-zero values **do**
 - 12: Create a dense matrix representing the block.
 - 13: Pad the block if it's smaller than the required size for wavelet decomposition.
 - 14: Perform 2D wavelet decomposition on the block.
 - 15: Resize the wavelet coefficients using exact resizing.
 - 16: Reconstruct the scaled block from the resized coefficients using inverse wavelet transform.
 - 17: Calculate the starting row and column indices for placing the scaled block in the new matrix.
 - 18: **for** each element in the reconstructed block **do**
 - 19: **if** the element's value is above a threshold **then**
 - 20: Add the element's row and column indices and value to the result lists. **if**
 - 21: **end for**
 - 22: **end for**
 - 23: Create a sparse matrix in CSR format from the collected row and column indices and values.
 - 24: **return** the scaled sparse matrix.
-

Algorithm 14 Scale Sparse Matrix with DCT (Blockwise)

Require: Original sparse matrix, New size, Block size (optional), Threshold (optional)

Ensure: Resized sparse matrix

- 1: Get original matrix dimensions.
 - 2: Calculate scaling factor.
 - 3: Get non-zero row and column indices and values from the original matrix.
 - 4: Initialize an empty dictionary to store blocks of non-zero values.
 - 5: **for** each non-zero element in the original matrix **do**
 - 6: Determine the block containing the element.
 - 7: Store the element's position and value within the block.
 - 8: **end for**
 - 9: Initialize lists to store the row and column indices and values of the scaled matrix.
 - 10: **for** each block of non-zero values **do**
 - 11: Create a dense matrix representing the block.
 - 12: Calculate new block dimensions based on the scaling factor.
 - 13: **if** the block is not empty **then**
 - 14: Perform 2D DCT on the block.
 - 15: Resize the DCT coefficients (crop/pad) to the new block dimensions.
 - 16: Perform inverse 2D DCT on the resized coefficients.
 - 17: **for** each element in the resized block **do**
 - 18: **if** the element's value is above the threshold **then**
 - 19: Calculate the element's position in the new matrix.
 - 20: **if** the element is within the new matrix boundaries **then**
 - 21: Add the element's position and value to the result lists. **if**
 - 22: **end for**
 - 23: **end for**
 - 24: Create a sparse matrix in CSR format from the collected row and column indices and values.
 - 25: **return** the resized sparse matrix.
-

Algorithm 15 Scale Sparse Matrix with Fourier

Require: Original sparse matrix, New size

Ensure: Scaled sparse matrix

- 1: Convert the sparse matrix to a dense array.
 - 2: Perform 2D forward FFT on the dense array.
 - 3: Shift the frequency spectrum to the center.
 - 4: Create a new array with the target shape, filled with zeros.
 - 5: Copy the central portion of the shifted spectrum to the new array.
 - 6: Perform inverse shift on the padded/cropped spectrum.
 - 7: Perform inverse 2D FFT on the shifted spectrum.
 - 8: Take the real part of the result.
 - 9: Threshold small values in the result to zero.
 - 10: Convert the thresholded result to a sparse matrix (CSR format).
 - 11: **return** the scaled sparse matrix.
-

Algorithm 16 Scale Sparse Matrix with Graph

Require: Original sparse matrix, New size, Match non-zero count (optional)

Ensure: Scaled sparse matrix

- 1: Convert the original matrix to CSR format.
 - 2: Store the original matrix size.
 - 3: **if** New size $\neq$ Original size **then**
 - 4: **while** current matrix size $\neq$ New size **do**
 - 5: Coarsen the matrix using heavy-edge matching. **While**
 - 6: **if** current matrix size $\neq$ New size **then**
 - 7: Sample nodes by strength to reach the New size. **if** New size $\neq$ Original size **then**
 - 8: **while** current matrix size $\neq$ New size **do**
 - 9: Refine the matrix using weight-aware edge duplication. **While if**
 - 10: **if** Match non-zero count is true **then**
 - 11: Get the current number of non-zero elements.
 - 12: Calculate the desired number of non-zero elements. **if** Current non-zero count $\neq$ Desired non-zero count **then**
 - 13: Randomly remove non-zero elements to match the desired count. **elseif** Current non-zero count $\neq$ Desired non-zero count **then**
 - 14: Add random non-zero elements to match the desired count. **if if**
 - 15: **return** the scaled sparse matrix.
-

Algorithm 17 Scale Sparse Matrix with Kronecker

Require: Original sparse matrix, New size, Output path, Match non-zero count (optional), Initiator size (optional), Random seed (optional)

Ensure: Scaled sparse matrix

- 1: **if** original matrix is not square **then**
 - 2: Raise an error.
 - 3: Initialize a random number generator.
 - 4: Fit initiator matrix via block-averaging.
 - 5: Calculate the number of Kronecker products needed.
 - 6: Renormalize the initiator matrix to match the original density.
 - 7: Expand the initiator matrix using Kronecker products.
 - 8: **if** expanded matrix is larger than the target size **then**
 - 9: Crop the expanded matrix to the target size. % End if
 - 10: **if** match non-zero count is true **then**
 - 11: Calculate the target number of non-zero elements.
 - 12: Get the current number of non-zero elements in the expanded matrix. **if** Current non-zero count $\neq$ Target non-zero count **then**
 - 13: Randomly drop surplus edges. **elseif** Current non-zero count $\neq$ Target non-zero count **then**
 - 14: Randomly add edges without duplicating existing ones.
 - 15: Save the scaled matrix to the output path.
 - 16: **return** the scaled sparse matrix. =0
-